

Sommaire

1. Introduction	p.1
1.1 Contexte du TP	
1.2 Qu'est-ce que Kali Linux ?	
2. Architecture du TP	p.2
2.1 Topologie réseau	
2.2 Principe Attaque/Défense	
3. Brute Force FTP et Protection avec Fail2Ban	p.3
3.1 Contexte	
3.2 Phase d'attaque : Brute Force FTP	
3.3 Phase de défense : Déploiement de Fail2Ban	
3.4 Test de la protection	
4. Attaque par Déni de Service (DoS)	p.8
4.1 Contexte	
4.2 Phase d'attaque : Simulation DoS	
4.3 Phase de défense : Protection anti-DoS	
4.4 Test de la protection	
5. Déploiement et Sécurisation d'un Serveur SIP avec Asterisk	p.10
5.1 Contexte	
5.2 Installation et configuration d'Asterisk	
5.3 Configuration du serveur SIP	
5.4 Phase d'attaque : Vulnérabilités SIP	
5.5 Phase de défense : Sécurisation d'Asterisk	
6. Scan de ports et énumération de services avec Nmap	p.16
6.1 Contexte	
6.2 Phase d'attaque : Reconnaissance avec Nmap	

6.3 Phase de défense : Protection contre le scan

6.4 Enseignements

7. Sniffing réseau et capture de credentials FTP p.17

7.1 Contexte

7.2 Phase d'attaque : Capture de trafic avec Wireshark/tcpdump

7.3 Phase de défense : Chiffrement des communications

7.4 Enseignements

8. Annexes (captures) p.19

9. Glossaire p.24

10. Conclusion p.26

1. Introduction

1.1 Contexte du TP

L'objectif principal de ce TP est de comprendre les mécanismes d'attaque et de défense dans un environnement réseau contrôlé, en adoptant une approche "Red Team / Blue Team".

1.2 Qu'est-ce que Kali Linux ?

Kali Linux est une distribution Linux basée sur Debian, spécialement conçue pour les tests de pénétration (penetration testing) et l'audit de sécurité informatique. Développée et maintenue par Offensive Security, elle constitue la référence dans le domaine de la cybersécurité offensive.

Caractéristiques principales :

- ⑩ Plus de 600 outils préinstallés : scanners de vulnérabilités, outils de cracking, sniffers réseau, frameworks d'exploitation, etc.
- ⑩ Environnement personnalisable : adaptable selon les besoins des professionnels de la sécurité
- ⑩ Support multi-plateforme : disponible en versions pour ordinateurs, ARM, cloud et conteneurs
- ⑩ Documentation complète : ressources étendues pour l'apprentissage et la pratique éthique du hacking

Utilisation professionnelle :

Kali Linux est utilisé par les experts en sécurité pour évaluer la robustesse des systèmes informatiques, identifier les vulnérabilités, tester les mécanismes de défense et former les administrateurs aux bonnes pratiques de sécurité. Dans un contexte pédagogique comme le BTS SIO, il permet de comprendre concrètement les menaces auxquelles sont confrontées les infrastructures modernes.

2. Architecture du TP

2.1 Topologie réseau

L'infrastructure mise en place pour ce TP comprend deux machines virtuelles sur un réseau local isolé :

Machine	Système d'exploitation	Adresse IP	Rôle
Machine attaquante	Kali Linux	172.20.2.50/24	Client
Machine cible	Ubuntu Server	172.20.2.25/24	Serveur

Réseau : 172.20.2.0/24
Passerelle : Non spécifiée (réseau isolé)

2.2 Principe Attaque/Défense

Le TP suit une méthodologie en deux phases :

1. Phase d'attaque : exploitation des vulnérabilités depuis Kali Linux
2. Phase de défense : mise en place de contre-mesures sur Ubuntu Server

Cette approche permet de comprendre à la fois les techniques d'intrusion et les solutions de protection adaptées.

3. Brute Force FTP et Protection avec Fail2Ban

3.1 Contexte

Le protocole FTP (File Transfer Protocol) reste largement utilisé pour le transfert de fichiers. Cependant, son mécanisme d'authentification par login/mot de passe le rend vulnérable aux attaques par force brute, où un attaquant tente de deviner les identifiants en testant de nombreuses combinaisons.

3.2 Phase d'attaque : Brute Force FTP

Outils utilisés

Hydra : outil de cracking d'identifiants réseau

Service cible : vsftpd (Very Secure FTP Daemon)

Méthodologie

L'attaque par force brute consiste à tester systématiquement des combinaisons de noms d'utilisateur et de mots de passe jusqu'à trouver les bons identifiants. Cette technique est particulièrement efficace contre les comptes utilisant des mots de passe faibles ou par défaut.

Commande type d'attaque

bash

```
hydra -l <utilisateur> -P <wordlist> ftp://172.20.2.25
```

Paramètres :

-l : spécifie le nom d'utilisateur

-P : fichier contenant la liste de mots de passe à tester

Protocole et cible : ftp://172.20.2.25

Résultats observés

L'attaque génère de nombreuses tentatives de connexion sur le serveur FTP, créant des logs d'authentification échouée. Sans protection, un attaquant patient peut éventuellement découvrir les identifiants valides.

3.3 Phase de défense : Déploiement de Fail2Ban

Qu'est-ce que Fail2Ban ?

Fail2Ban est un framework de prévention des intrusions qui analyse les fichiers journaux (logs) du système et bannit automatiquement les adresses IP présentant un comportement malveillant (tentatives de connexion répétées, scans de ports, etc.).

Installation et configuration

D'après les captures d'écran fournies, la configuration de Fail2Ban a été réalisée comme suit :

1. Édition du fichier de configuration principal

bash

```
sudo nano /etc/fail2ban/jail.local
```

2. Configuration des sections de protection

La configuration montre plusieurs "jails" (prisons) définies pour protéger différents services :

Protection FTP (proftpd)

```
[proftpd]
```

```
enabled = false
```

```
port = ftp,ftp-data,ftps,ftps-data
```

```
filter = proftpd
```

```
logpath = /var/log/proftpd/proftpd.log
```

```
maxretry = 3
```

Protection vsftpd (activée)

```
[recidive]
```

```
enabled = true
```

```
filter = recidive
```

```
logpath = /var/log/fail2ban.log
```

```
banaction = iptables-allports
```

```
bantime = 86400
```

```
findtime = 86400
```

Paramètres clés :

enabled = true : active la protection

port : ports surveillés

filter : expressions régulières pour détecter les tentatives d'intrusion

logpath : emplacement des logs à surveiller

bantime : durée du bannissement (86400 secondes = 24 heures)

findtime : fenêtre temporelle d'analyse

maxretry : nombre maximum de tentatives avant bannissement (3 essais)

Autres protections configurées

Le fichier montre également la configuration pour :

apache-noscript : protection contre les scripts malveillants Apache

apache-overflows : protection contre les débordements de buffer Apache

nginx-http-auth : protection de l'authentification Nginx

apache-auth : protection de l'authentification Apache

apache-badbots : blocage des robots malveillants

Mécanisme de fonctionnement

1. Fail2Ban surveille en permanence les logs spécifiés
2. Lorsqu'une IP dépasse le seuil de maxretry dans la période findtime
3. L'IP est automatiquement bannie pour la durée bantime
4. Le bannissement est effectué via iptables (pare-feu Linux)
5. Les récidivistes sont identifiés et bannis pour des durées plus longues

Vérification du fonctionnement

Après configuration, le service a été activé :

bash

```
sudo systemctl start fail2ban
```

```
sudo systemctl enable fail2ban
```

```
sudo systemctl status fail2ban
```

Le statut montre que le service est active (running) depuis le 10 novembre 2025 à 14:47:54 CET.

3.4 Test de la protection

Lors d'une nouvelle tentative d'attaque par force brute depuis Kali Linux, Fail2Ban détecte les tentatives répétées et bannit automatiquement l'adresse IP 172.20.2.50, rendant l'attaque inefficace. L'attaquant ne peut plus se connecter au serveur FTP, même avec les bons identifiants.

4. Attaque par Déni de Service (DoS)

4.1 Contexte

Une attaque par déni de service (DoS - Denial of Service) vise à rendre un service ou une machine indisponible en saturant ses ressources (bande passante, CPU, mémoire, connexions) ou en exploitant des vulnérabilités du protocole réseau.

4.2 Phase d'attaque : Simulation DoS

Principes de l'attaque DoS

Plusieurs techniques peuvent être utilisées :

SYN Flood : envoi massif de requêtes TCP SYN sans compléter le handshake

UDP Flood : inondation de paquets UDP sur des ports aléatoires

ICMP Flood (Ping Flood) : envoi massif de requêtes ping

HTTP Flood : requêtes HTTP massives pour saturer un serveur web

Outils potentiellement utilisés

Depuis Kali Linux, plusieurs outils permettent de réaliser des tests DoS :

hping3 : générateur de paquets personnalisés

LOIC (Low Orbit Ion Cannon) : outil d'attaque DoS

slowloris : attaque DoS applicative sur serveurs web

Exemple de commande (SYN Flood)

bash

```
hping3 -S --flood -V -p 80 172.20.2.25
```

Paramètres :

-S : envoi des paquets SYN

--flood : mode inondation (envoi maximum)

-V : mode verbeux

-p 80 : cible le port HTTP

Impact observé

Sur la machine cible (172.20.2.25), on observe :

Augmentation drastique de la charge CPU

Saturation de la bande passante réseau

Ralentissement ou indisponibilité des services

Logs système remplis de tentatives de connexion

4.3 Phase de défense : Protection anti-DoS

Mesures au niveau du pare-feu (iptables)

1. Limitation du taux de connexions SYN

bash

```
iptables -A INPUT -p tcp --syn -m limit --limit 1/s --limit-burst 3 -j ACCEPT
```

```
iptables -A INPUT -p tcp --syn -j DROP
```

Cette règle limite les nouvelles connexions TCP à 1 par seconde avec une tolérance de 3 connexions simultanées.

2. Protection contre le SYN Flood via sysctl

bash

```
sysctl -w net.ipv4.tcp_syncookies=1
```

```
sysctl -w net.ipv4.tcp_max_syn_backlog=2048
```

```
sysctl -w net.ipv4.tcp_synack_retries=2
```

3. Limitation des requêtes ICMP (ping)

bash

```
iptables -A INPUT -p icmp --icmp-type echo-request -m limit --limit 1/s -j ACCEPT  
iptables -A INPUT -p icmp --icmp-type echo-request -j DROP
```

Utilisation de Fail2Ban pour DoS

Fail2Ban peut également détecter et bloquer les attaques DoS en analysant les logs et en identifiant les comportements anormaux. Une configuration spécifique peut être ajoutée pour bannir les IP générant un trafic excessif.

Solutions au niveau applicatif

Nginx/Apache : modules de limitation du taux de requêtes (mod_evasive, limit_req)

CloudFlare ou autres CDN : protection DDoS professionnelle

Reverse proxy : filtrage et distribution de charge

4.4 Test de la protection

Les paquets malveillants sont filtrés ou limités

Le serveur reste accessible pour les utilisateurs légitimes

La charge système reste maîtrisée

Les tentatives d'attaque sont journalisées

5. Déploiement et Sécurisation d'un Serveur SIP avec Asterisk

5.1 Contexte

Le protocole SIP (Session Initiation Protocol) est utilisé pour établir, maintenir et terminer des sessions de communication en temps réel (VoIP, visioconférence). Asterisk est une solution open-source de serveur téléphonique (IPBX) qui implémente ce protocole.

5.2 Installation et configuration d'Asterisk

Téléchargement et extraction

le processus d'installation a suivi ces étapes :

1. Téléchargement de la source

bash

cd /tmp

wget https://downloads.asterisk.org/pub/telephony/asterisk/asterisk-20-current.tar.gz

2. Extraction de l'archive

bash

tar -xzf asterisk-20-current.tar.gz

cd asterisk-20*/

3. Installation des dépendances

bash

sudo contrib/scripts/get_mp3_source.sh

sudo contrib/scripts/install_prereq install

Le script `install_prereq` installe automatiquement toutes les dépendances nécessaires :

`aptitude-common, libcwwidget, libxapian30`

`aptitude-doc-en, aptitude-doc, debtags`

`libcwwidget-dev, xapian-tools`

4. Configuration de la compilation

`bash`

`sudo ./configure --with-jansson-bundled --with-pjproject-bundled`

Options importantes :

`--with-jansson-bundled` : inclut la bibliothèque JSON Jansson

`--with-pjproject-bundled` : inclut PJSIP (stack SIP moderne)

Le script de configuration vérifie :

Le système de compilation (`x86_64-pc-linux-gnu`)

La présence de GCC

Les fonctionnalités du compilateur C

Le support des objets de suffixe `.o`

Les capacités de cross-compilation

5. Compilation et installation

`bash`

`sudo make`

`sudo make install`

`sudo ldconfig`

`sudo systemctl start asterisk`

`sudo systemctl enable asterisk`

...

Vérification de l'installation

Le statut du service montre qu'Asterisk est correctement installé :

...

● `asterisk.service` - LSB: Asterisk PBX

Loaded: loaded (/etc/init.d/asterisk; generated)

Active: active (running) since Mon 2025-11-10 14:47:54 CET; 9s ago

5.3 Configuration du serveur SIP

Fichier sip.conf

Le fichier de configuration `/etc/asterisk/sip.conf` a été créé/modifié :

bash

```
sudo cp /etc/asterisk/sip.conf /etc/asterisk/sip.conf.bak
```

```
sudo rm /etc/asterisk/sip.conf
```

```
sudo nano /etc/asterisk/sip.conf
```

Une sauvegarde a d'abord été créée pour pouvoir revenir à la configuration par défaut si nécessaire.

Structure type du fichier sip.conf :

```
[general]
```

```
context=default
```

```
allowoverlap=no
```

```
bindport=5060
```

```
bindaddr=0.0.0.0
```

```
srvlookup=yes
```

```
disallow=all
```

```
allow=ulaw
```

```
allow=alaw
```

```
allow=gsm
```

```
[1001]
```

```
type=friend
```

```
context=from-internal
```

```
host=dynamic
```

```
secret=motdepasse201
```

```
dtmfmode=rfc2833
```

```
[1002]
```

```
type=friend
```

```
context=from-internal
```

```
host=dynamic
```

```
secret=motdepasse202
```

```
dtmfmode=rfc2833
```

`bindport=5060` : port SIP standard
`bindaddr=0.0.0.0` : écoute sur toutes les interfaces
`allow=ulaw/alaw/gsm` : codecs audio autorisés
`type=friend` : le client peut émettre et recevoir
`host=dynamic` : IP dynamique (enregistrement requis)
`secret` : mot de passe du compte SIP

Configuration sur l'interface web des téléphones, on a configuré 2 téléphones un en 1001, l'autre en 1002 :

Non sécurisé 172.20.2.61/globalSIPsettings.html Résumer 675 Log

Status
System Information
License Status

Operation
User Password
Phone Lock
Programmable Keys
Keypad Speed Dial
Directory
Reset

Basic Settings
Preferences
Account Configuration

Advanced Settings
Network
Global SIP
Line 1
Line 2
Line 3
Line 4
Line 5
Line 6
Line 7
Line 8
Line 9
Action URI
Configuration Server
Firmware Update
TLS Support
802.1x Support
Troubleshooting

Global SIP Settings

Basic SIP Authentication Settings

Screen Name	1002
Screen Name 2	
Phone Number	1002
Caller ID	1002
Authentication Name	1002
Password	****
BLA Number	
Line Mode	Generic
Call Waiting	☒ Enabled

Basic SIP Network Settings

Proxy Server	0.0.0.0
Proxy Port	0
Backup Proxy Server	0.0.0.0
Backup Proxy Port	0
Outbound Proxy Server	0.0.0.0
Outbound Proxy Port	0
Backup Outbound Proxy Server	0.0.0.0
Backup Outbound Proxy Port	0
Registrar Server	0.0.0.0
Registrar Port	0
Backup Registrar Server	0.0.0.0
Backup Registrar Port	0
Registration Period	0
Conference Server URI	

5.4 Phase d'attaque : Vulnérabilités SIP

Types d'attaques possibles

1. Énumération des comptes SIP

Outil : svmap (SIPVicious)

Objectif : découvrir les extensions actives

bash

svmap 172.20.2.25

2. Attaque par force brute

Outil : svcrack (SIPVicious)

Objectif : deviner les mots de passe des comptes

```
svcrack -u 201 -d dictionary.txt 172.20.2.25
```

3. Interception de communications (Man-in-the-Middle)

Outil : Wireshark, tcpdump

Risque : capture du trafic SIP non chiffré (RTP)

4. Dénî de service SIP

Inondation de requêtes INVITE, REGISTER

Saturation des ressources du serveur Asterisk

5. SIP Spoofing

Falsification de l'identité appelante

Usurpation d'identité pour passer des appels

5.5 Phase de défense : Sécurisation d'Asterisk

Protection via Fail2Ban (déjà en place)

La configuration Fail2Ban peut être étendue pour protéger Asterisk :

```
[asterisk]
enabled = true
port = 5060,5061
filter = asterisk
logpath = /var/log/asterisk/full
maxretry = 5
bantime = 86400
```

Le filtre détecte :

Tentatives d'enregistrement échouées

Tentatives d'authentification multiples

Scans d'extensions

Bonnes pratiques de sécurisation

1. Mots de passe robustes

```
secret=Tr0ng_P@ssw0rd_2025!
```

2. Limitation par adresse IP

```
[1001][1002]
type=friend
host=172.20.2.50
secret=motdepasse201
```

3. Désactivation des comptes invités

```
allowguest=no
```

4. Utilisation du chiffrement (TLS/SRTP)

```
tlsenable=yes  
tlsbindaddr=0.0.0.0:5061  
tlscertfile=/etc/asterisk/keys/asterisk.pem  
tlsprivatekey=/etc/asterisk/keys/asterisk.key
```

5. Pare-feu restrictif

bash

Autoriser uniquement les IP de confiance sur le port SIP

```
iptables -A INPUT -p udp --dport 5060 -s 172.20.2.0/24 -j ACCEPT
```

```
iptables -A INPUT -p udp --dport 5060 -j DROP
```

Ports RTP pour la voix

```
iptables -A INPUT -p udp --dport 10000:20000 -s 172.20.2.0/24 -j ACCEPT
```

6 . Scan de ports et énumération de services avec Nmap

Contexte

Avant toute attaque ciblée, un attaquant effectue une reconnaissance pour identifier les services actifs et leurs versions. Cette phase est cruciale car elle révèle les points d'entrée potentiels.

Phase d'attaque : Reconnaissance avec Nmap

Outils utilisés : Nmap (Network Mapper)

Commandes type :

bash

Scan des ports ouverts

```
nmap -sS -sV -O 172.20.2.25
```

Scan complet avec scripts de vulnérabilités

Paramètres :

-sS : scan SYN (furtif)
-sV : détection des versions de services
-O : détection du système d'exploitation
--script=vuln : exécution des scripts de détection de vulnérabilités

Résultats observés : L'attaquant obtient la liste des ports ouverts (21/FTP, 22/SSH, 5060/SIP), les versions exactes des logiciels, et potentiellement des vulnérabilités connues (CVE).

Phase de défense : Protection contre le scan

1. Limitation avec iptables :

```
bash
# Détecter et bloquer les scans de ports
iptables -A INPUT -p tcp --tcp-flags ALL NONE -j DROP
iptables -A INPUT -p tcp --tcp-flags ALL ALL -j DROP
iptables -A INPUT -p tcp --tcp-flags SYN,FIN SYN,FIN -j DROP
```

2. Installation de portsentry :

```
bash
apt install portsentry
# Configure /etc/portsentry/portsentry.conf
# BLOCK_UDP="1"
# BLOCK_TCP="1"
systemctl enable portsentry
```

3. Fail2Ban pour Nmap :

```
ini
[portscan]
enabled = true
filter = portscan
logpath = /var/log/syslog
maxretry = 3
bantime = 86400
```

Enseignements

La reconnaissance est la première étape de toute attaque. Limiter l'information exposée et détecter les scans permet de ralentir significativement un attaquant.

7 . Sniffing réseau et capture de credentials FTP

Contexte

Le protocole FTP transmet les identifiants en clair. Sur un réseau local, un attaquant peut intercepter le trafic et récupérer les mots de passe sans les deviner.

Phase d'attaque : Capture de trafic avec Wireshark/tcpdump

Outils utilisés : Wireshark, tcpdump, ettercap

Méthodologie :

bash

```
# Capture du trafic FTP sur l'interface réseau  
tcpdump -i eth0 port 21 -w capture_ftp.pcap
```

```
# Ou avec Wireshark (interface graphique)  
wireshark &  
# Filtre : ftp
```

Analyse de la capture : Dans Wireshark, filtrer avec `ftp.request.command == "USER" || ftp.request.command == "PASS"` révèle les identifiants transmis en clair.

Résultats observés : Les credentials FTP (utilisateur/mot de passe) sont visibles directement dans les paquets capturés, sans nécessité de brute force.

Phase de défense : Chiffrement des communications

1. Remplacer FTP par SFTP (SSH) :

bash

```
# Désactiver FTP  
systemctl stop vsftpd
```

Utiliser SSH/SFTP (déjà sécurisé)
Les utilisateurs se connectent via *sftp user@172.20.2.25*

2. Si FTP requis, activer FTPS (FTP over TLS) :

bash

Dans /etc/vsftpd.conf
ssl_enable=YES
rsa_cert_file=/etc/ssl/certs/vsftpd.pem
rsa_private_key_file=/etc/ssl/private/vsftpd.key
force_local_data_ssl=YES
force_local_logins_ssl=YES

3. Segmentation réseau (VLAN) : Isoler les serveurs critiques dans un VLAN séparé empêche le sniffing depuis les postes utilisateurs.

4. Détection du sniffing (mode promiscuous) :

bash

Vérifier si une interface est en mode promiscuous
ip link show | grep PROMISC

Script de détection
apt install promisc-detect

Enseignements

Les protocoles en clair (FTP, Telnet, HTTP) sont vulnérables à l'interception. Le chiffrement (SFTP, FTPS, HTTPS) est indispensable pour protéger les credentials en transit.

7 . Annexes (capture)

Capture Brute force FTP

```
GNU nano 7.2 /etc/vsftpd.conf
#
# This directive enables listening on IPv6 sockets. By default, listening
# on the IPv6 "any" address (:::) will accept connections from both IPv6
# and IPv4 clients. It is not necessary to listen on *both* IPv4 and IPv6
# sockets. If you want that (perhaps because you want to listen on specific
# addresses) then you must run two copies of vsftpd with two configuration
# files.
listen_ipv6=YES
#
# Allow anonymous FTP? (Disabled by default).
anonymous_enable=NO
#
# Uncomment this to allow local users to log in.
local_enable=YES
#
# Uncomment this to enable any form of FTP write command.
write_enable=YES
```

```
root@admin1-virtualmachine:~# cd /etc/vsftpd.conf
-bash: cd: /etc/vsftpd.conf: N'est pas un dossier
root@admin1-virtualmachine:~# nano /etc/vsftpd.conf
root@admin1-virtualmachine:~# systemctl enable fail2ban
Synchronizing state of fail2ban.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable fail2ban
root@admin1-virtualmachine:~#
::1                ff02::2                ip6-localhost
admin1-virtualmachine ip6-allnodes          ip6-loopback
ff02::1            ip6-allrouters        localhost
root@admin1-virtualmachine:~#
```

GNU nano 7.2 /etc/fail2ban/jail.local

```
[DEFAULT]
bantime = 3600
findtime = 600
maxretry = 5
banaction = iptables-multiport
destemail = votre-email@exemple.com
sender = fail2ban@localhost
action = %(action_)s

[sshd]
enabled = true
port = ssh
filter = sshd
logpath = /var/log/auth.log
backend = systemd
maxretry = 3
bantime = 7200

[vsftpd]
enabled = true
port = ftp,ftp-data,ftps,ftps-data
filter = vsftpd
logpath = /var/log/vsftpd.log
maxretry = 3
bantime = 3600
```

GNU nano 7.2 /etc/fail2ban/jail.local

```
[proftpd]
enabled = false
port = ftp,ftp-data,ftps,ftps-data
filter = proftpd
logpath = /var/log/proftpd/proftpd.log
maxretry = 3

[apache-auth]
enabled = false
port = http,https
filter = apache-auth
logpath = /var/log/apache2/*error.log
maxretry = 5

[apache-badbots]
enabled = false
port = http,https
filter = apache-badbots
logpath = /var/log/apache2/*access.log
maxretry = 2
```

```
GNU nano 7.2 /etc/fail2ban/jail.local
[apache-noscript]
enabled = false
port = http,https
filter = apache-noscript
logpath = /var/log/apache2/*error.log

[apache-overflows]
enabled = false
port = http,https
filter = apache-overflows
logpath = /var/log/apache2/*error.log
maxretry = 2

[nginx-http-auth]
enabled = false
port = http,https
filter = nginx-http-auth
logpath = /var/log/nginx/error.log
```

```
[recidive]
enabled = true
filter = recidive
logpath = /var/log/fail2ban.log
banaction = iptables-allports
bantime = 86400
findtime = 86400
maxretry = 3
```

Capture Astérisk

```
root@dbquimarchepas:~# cd /tmp
root@dbquimarchepas:/tmp# wget https://downloads.asterisk.org/pub/telephony/asterisk/asterisk-20-current.tar.gz
--2025-11-10 14:40:53-- https://downloads.asterisk.org/pub/telephony/asterisk/asterisk-20-current.tar.gz
Résolution de downloads.asterisk.org (downloads.asterisk.org)... 2604:a880:400:d0::14:9001, 165.22.184.19
Connexion à downloads.asterisk.org (downloads.asterisk.org)|2604:a880:400:d0::14:9001|:443... connecté.
requête HTTP transmise, en attente de la réponse... 200 OK
Taille : 28540711 (27M) [application/octet-stream]
Sauvegarde en : « asterisk-20-current.tar.gz »

asterisk-20-current.tar.gz 100%[=====>] 27,22M 15,2MB/s ds 1,8s
2025-11-10 14:40:55 (15,2 MB/s) – « asterisk-20-current.tar.gz » sauvegardé [28540711/28540711]
```

```
root@dbquimarchepas:/tmp# tar -xzf asterisk-20-current.tar.gz
asterisk-20.16.0/
asterisk-20.16.0/.cleancount
asterisk-20.16.0/.gitignore
asterisk-20.16.0/.version
asterisk-20.16.0/BSDmakefile
asterisk-20.16.0/BUGS
asterisk-20.16.0/CHANGES.html
asterisk-20.16.0/CHANGES.md
asterisk-20.16.0/COPYING
asterisk-20.16.0/CREDITS
```

```
root@dbquimarchepas:/tmp# cd asterisk-20*/
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo contrib/scripts/get_mp3_source.sh
sudo contrib/scripts/install_prereq install
contrib/scripts/get_mp3_source.sh: 18: svn: not found
Lecture des listes de paquets... Fait
Construction de l'arbre des dépendances... Fait
Lecture des informations d'état... Fait
Les paquets supplémentaires suivants seront installés :
  aptitude-common libcwidget4 libxapian30
Paquets suggérés :
  apt-xapian-index aptitude-doc-en | aptitude-doc debtags libcwidget-dev xapian-tools
Les NOUVEAUX paquets suivants seront installés :
```

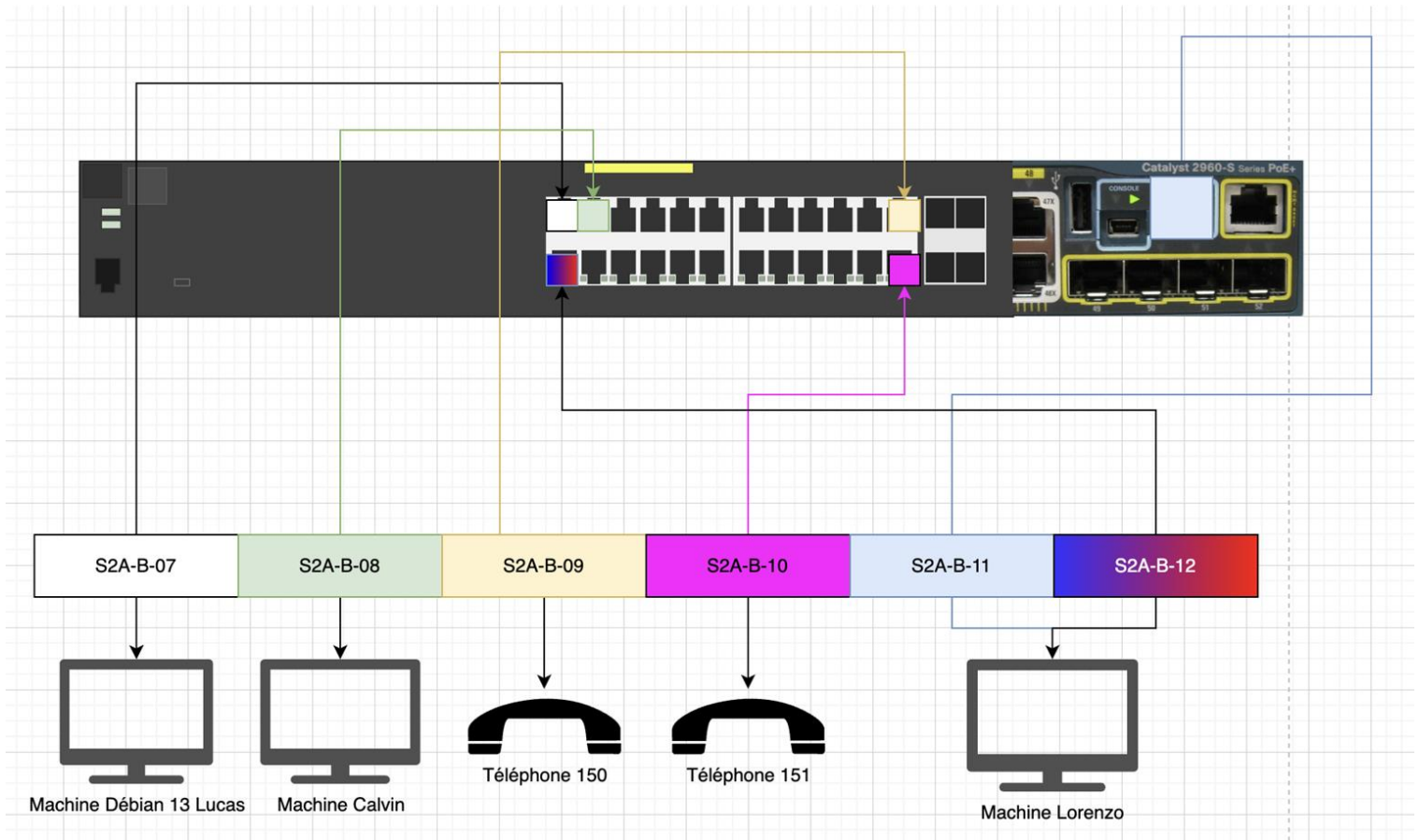
```
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo ./configure --with-jansson-bundled --with-pjproject-bundled
checking build system type... x86_64-pc-linux-gnu
checking host system type... x86_64-pc-linux-gnu
checking for gcc... gcc
checking whether the C compiler works... yes
checking for C compiler default output file name... a.out
checking for suffix of executables...
checking whether we are cross compiling... no
checking for suffix of object files... o
checking whether the compiler supports GNU C... yes
checking whether gcc accepts -g... yes
```

```
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo ldconfig
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo systemctl start asterisk
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo systemctl enable asterisk
asterisk.service is not a native service, redirecting to systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable asterisk
root@dbquimarchepas:/tmp/asterisk-20.16.0# sudo systemctl status asterisk
● asterisk.service - LSB: Asterisk PBX
   Loaded: loaded (/etc/init.d/asterisk; generated)
   Active: active (running) since Mon 2025-11-10 14:47:54 CET; 9s ago
 Invocation: 9b977c463651491bbd4f055cfd7d8e07
    Docs: man:systemd-sysv-generator(8)
   Tasks: 76 (limit: 4604)
  Memory: 52.7M (peak: 53.2M)
     CPU: 824ms
    CGroup: /system.slice/asterisk.service
            └─46307 /usr/sbin/asterisk

nov. 10 14:47:54 dbquimarchepas systemd[1]: Starting asterisk.service - LSB: Asterisk PBX...
nov. 10 14:47:54 dbquimarchepas asterisk[46294]: Starting Asterisk PBX: asterisk.
nov. 10 14:47:54 dbquimarchepas systemd[1]: Started asterisk.service - LSB: Asterisk PBX.

root@dbquimarchepas:~# sudo cp /etc/asterisk/sip.conf /etc/asterisk/sip.conf.bak
root@dbquimarchepas:~# rm /etc/asterisk/sip.conf
root@dbquimarchepas:~# sudo nano /etc/asterisk/sip.conf
```

Schémas du TP Astérisik



8. Glossaire

Asterisk : Logiciel open-source de serveur téléphonique (IPBX) permettant de gérer des communications VoIP via le protocole SIP.

Brute Force : Technique d'attaque consistant à tester systématiquement toutes les combinaisons possibles de mots de passe jusqu'à trouver le bon.

DoS (Denial of Service) : Attaque visant à rendre un service indisponible en saturant ses ressources.

DDoS (Distributed Denial of Service) : Attaque DoS menée depuis plusieurs sources simultanément.

Fail2Ban : Logiciel de prévention d'intrusion qui analyse les logs et bannit automatiquement les IP malveillantes.

Firewall (Pare-feu) : Système de sécurité filtrant le trafic réseau entrant et sortant selon des règles définies.

FTP (File Transfer Protocol) : Protocole de transfert de fichiers, vulnérable car transmettant les données en clair.

FTPS : Version sécurisée de FTP utilisant le chiffrement TLS/SSL.

Hydra : Outil de cracking d'identifiants réseau par force brute, capable de tester de nombreux protocoles.

ICMP (Internet Control Message Protocol) : Protocole réseau utilisé notamment par la commande ping.

iptables : Outil en ligne de commande permettant de configurer le pare-feu du noyau Linux.

Jail (Prison) : Dans Fail2Ban, configuration définissant les règles de surveillance et de bannissement pour un service spécifique.

Log (Journal) : Fichier enregistrant les événements système, utilisé pour la surveillance et le diagnostic.

Man-in-the-Middle (MitM) : Attaque où l'attaquant intercepte secrètement les communications entre deux parties.

Nmap (Network Mapper) : Outil de scan réseau permettant de découvrir les hôtes et services actifs.

Portsentry : Outil de détection et de blocage des scans de ports.

RTP (Real-time Transport Protocol) : Protocole de transport des flux audio/vidéo en temps réel.

SFTP (SSH File Transfer Protocol) : Protocole de transfert de fichiers sécurisé basé sur SSH.

SIP (Session Initiation Protocol) : Protocole de signalisation pour établir des sessions de communication multimédia (VoIP).

SRTP (Secure RTP) : Version chiffrée du protocole RTP pour sécuriser les communications vocales.

SYN Flood : Attaque DoS exploitant le processus de connexion TCP en envoyant massivement des paquets SYN.

Sysctl : Commande Linux permettant de modifier les paramètres du noyau à chaud.

TCP (Transmission Control Protocol) : Protocole de transport fiable, orienté connexion.

TLS (Transport Layer Security) : Protocole cryptographique assurant la confidentialité des communications.

UDP (User Datagram Protocol) : Protocole de transport rapide mais non fiable, sans établissement de connexion.

VLAN (Virtual Local Area Network) : Réseau local virtuel permettant de segmenter logiquement un réseau physique.

VoIP (Voice over IP) : Technologie de transmission de la voix sur les réseaux IP.

vsftpd (Very Secure FTP Daemon) : Serveur FTP sécurisé pour systèmes Unix/Linux.

Vulnérabilité : Faiblesse dans un système pouvant être exploitée par un attaquant.

Wireshark : Analyseur de protocoles réseau avec interface graphique, utilisé pour capturer et analyser le trafic.

10. Conclusion

Ce travail pratique a permis d'appréhender concrètement les enjeux de la cybersécurité à travers une approche pédagogique. En alternant entre les phases d'attaque depuis Kali Linux et les phases de défense sur Ubuntu Server, nous avons pu comprendre les mécanismes d'intrusion et les contre-mesures appropriées.

Les différentes manipulations réalisées ont démontré plusieurs points essentiels :

Sur le plan offensif, nous avons constaté que les infrastructures mal configurées sont particulièrement vulnérables aux attaques classiques : brute force FTP via Hydra, déni de service avec hping3, énumération de services avec Nmap, et interception de credentials en clair avec Wireshark. Ces techniques, bien que connues, restent redoutablement efficaces contre des systèmes non protégés.

Sur le plan défensif, l'implémentation de solutions de sécurité telles que Fail2Ban, le durcissement des règles iptables, le chiffrement des protocoles (SFTP, FTPS, TLS/SRTP) et la segmentation réseau ont prouvé leur efficacité pour contrer ces menaces. Le déploiement et la sécurisation du serveur VoIP Asterisk ont particulièrement illustré l'importance d'une approche de sécurité en profondeur combinant plusieurs couches de protection.

Enseignements clés :

- ⑩ La sécurité par défaut est insuffisante ; une configuration rigoureuse est indispensable
- ⑩ Les protocoles en clair (FTP, HTTP, Telnet) doivent être remplacés par leurs équivalents chiffrés
- ⑩ La surveillance active des logs et le bannissement automatique des comportements suspects sont cruciaux
- ⑩ La limitation des ressources et le filtrage du trafic permettent de contrer efficacement les attaques DoS

- ⑩ Une politique de mots de passe robustes combinée à une restriction par IP renforce significativement la sécurité